

Simulador de Planificación de Procesos

En la arquitectura de cualquier Sistema Operativo moderno, la multiprogramación y la gestión del procesador representan el pilar fundamental para maximizar el rendimiento del hardware. Dado que la CPU es un recurso secuencial único, la ilusión de ejecución simultánea de múltiples tareas depende por completo del despachador (dispatcher) y de las políticas algorítmicas del planificador a corto plazo.

Este simulador interactivo ha sido conceptualizado y desarrollado como una herramienta de análisis científico-práctico. Su propósito principal es modelar, trazar y contrastar el comportamiento de las seis principales políticas de planificación de hilos y procesos. El software trasciende el mero cálculo matemático pasivo: expone dinámicamente las fluctuaciones en el estado de los procesos y calcula las métricas de rendimiento estandarizadas para proporcionar criterios de diseño empíricos aplicables al desarrollo de núcleos de sistemas operativos.

Desglose Estructural de la Interfaz Gráfica y Máquina de Estados

La interfaz de usuario ha sido estructurada de forma modular para reflejar la transferencia de control y los cambios de contexto en tiempo real. Cada componente visual se corresponde directamente con una estructura de datos abstracta gestionada por el motor lúdico del simulador:

Panel Lateral de Control Global (Sidebar)

Ubicado a la izquierda de la pantalla, centraliza la mutación de las variables de entorno de la simulación. Permite seleccionar dinámicamente la política algorítmica activa, controlar la velocidad del reloj maestro del sistema mediante retardos ajustables en milisegundos (ms) por cada ciclo o tick, e inyectar vectores de prueba. La opción 'Randomize Dataset' puebla el vector mediante modelos aleatorios complejos, mientras que 'Add Manual Process' actúa como interfaz de interrupción externa para definir casos límite definidos por el usuario.

Tablero de Métricas de Rendimiento Acumulado

Dispuesto en el margen superior, actúa como un panel analítico que calcula de manera incremental los agregados estadísticos del lote completo. Presenta de forma transparente el tiempo medio de espera, el tiempo de retorno medio, el tiempo de respuesta inicial, el porcentaje de utilización de la CPU y el rendimiento por unidad de tiempo (Throughput).

Visualizador Dinámico de Colas y Núcleo de Procesamiento (Process Queues & CPU)

- **Ready Queue (Cola de Listos):** Representa la estructura de datos lineal (FIFO o cola de prioridades) donde residen los descriptores de procesos (PCB) que han realizado su transición de llegada al sistema y aguardan por la asignación de un cuanto o ráfaga de CPU.
- **CPU Estado (Running):** Celda monoprocesador que expone gráficamente cuál es el subconjunto de datos que mantiene el control exclusivo de la unidad aritmética lógica del procesador durante el tick vigente.
- **Blocked Queue (Cola de Bloqueados por E/S):** Estructura dedicada a contener aquellos procesos que, durante su ráfaga, solicitaron una llamada al sistema (system call) para operaciones asíncronas de Entrada/Salida (I/O). Al ingresar aquí, liberan la CPU de inmediato de forma voluntaria, modelando el comportamiento realista de los sistemas informáticos.

Línea de Tiempo Histórica (Diagrama de Gantt)

El componente GanttVisualizer representa de forma acumulativa la traza de ejecución cronológica de la CPU. Utiliza una codificación visual clara: las franjas de colores sólidos identifican el consumo neto de ciclos del procesador, mientras que las secciones con entramados o líneas diagonales señalizan los ciclos en los que el proceso estuvo retenido en operaciones de Entrada/Salida, permitiendo auditar con un solo vistazo la eficiencia de la política respecto a la alternancia de ráfagas.

Tabla Analítica de Detalles de Procesos Activos

Ubicada en la base del dashboard, expone los valores internos del PCB de cada proceso: Identificador (PID), instante de arribo, nivel de prioridad entera, ráfaga remanente versus total, y los cálculos individuales en curso de tiempos de retorno, respuesta y espera.

Algoritmos de Planificación Modelados

Los siete algoritmos implementados se dividen según su naturaleza de expropiación en No Expulsivos (Non-Preemptive) y Expulsivos/Apropiativos (Preemptive):

Algoritmo de Planificación	Naturaleza Operativa	Criterio de Selección de CPU	Riesgo de Inanición
FCFS (First-Come, First-Served)	No Expulsivo	Orden cronológico de llegada	Nulo

Algoritmo de Planificación	Naturaleza Operativa	Criterio de Selección de CPU	Riesgo de Inanición
SJF (Shortest Job First)	No Expulsivo	Menor longitud de ráfaga total de CPU	Alto (procesos largos)
SRTF (Shortest Remaining Time First)	Expulsivo (Apropiativo)	Menor tiempo de ráfaga remanente	Alto (procesos largos)
Round Robin (Turno Rotatorio)	Expulsivo (Apropiativo)	Orden FIFO limitado por Cuanto (Quantum)	Nulo
Priority (Non-Preemptive)	No Expulsivo	Mayor nivel de prioridad externa asignada	Alto (prioridades bajas)
Priority (Preemptive)	Expulsivo (Apropiativo)	Mayor prioridad instantánea en cola de listos	Alto (prioridades bajas)
Random (Aleatorio)	No Expulsivo (generalmente)	Selección estocástica / pseudoaleatoria	Alto (por probabilidad)

FCFS (First-Come, First-Served)

Política estrictamente secuencial regida por una cola FIFO estructurada. El despachador asigna la CPU al proceso cuyo tiempo de llegada sea el menor en el eje cronológico. Al ser de naturaleza no expulsiva, garantiza que el proceso retendrá el procesador hasta agotar por completo su ráfaga o requerir un bloqueo de E/S. Presenta como desventaja el denominado Efecto Convoy, donde procesos de ráfagas masivas degradan el tiempo de espera de tareas cortas subsiguientes.

SJF (Shortest Job First)

Algoritmo óptimo no expulsivo que reduce al mínimo el tiempo de espera promedio general del sistema. Examina los procesos en la cola de listos y otorga precedencia absoluta a aquel que requiera la ráfaga de CPU de menor envergadura. Si un proceso de ráfaga muy prolongada ingresa al sistema, sufrirá el fenómeno de inanición (starvation) si la cola es continuamente alimentada con procesos efímeros.

SRTF (Shortest Remaining Time First)

Constituye la variante expulsiva de SJF. En cada tick de reloj del simulador, el planificador reevalúa si un proceso recién llegado posee un tiempo de ráfaga remanente inferior al tiempo restante del proceso que se encuentra actualmente en ejecución. De ser así, se ejecuta una interrupción forzada, el proceso activo regresa a la Cola de Listos preservando su avance, y se realiza un cambio de contexto inmediato.

Round Robin (RR)

Algoritmo cíclico diseñado específicamente para sistemas de tiempo compartido. Se basa en una cola FIFO combinada con una interrupción basada en hardware por rebanada de tiempo fija denominada Quantum. El proceso activo dispone de la CPU únicamente durante esa fracción de tiempo. Al expirar el quantum, el planificador lo remueve de forma expulsiva, situándolo al final de la cola de listos. Es inmune a la inanición, pero altamente sensible a la dimensión del Quantum.

Prioridad No Expropiativa (Priority NP)

Asocia a cada proceso un número entero que cuantifica su rango de importancia dentro del sistema (donde valores numéricos bajos denotan una mayor prioridad). El planificador ordena la cola de listos en base a este ordenamiento jerárquico. El proceso con mayor prioridad se adueña de la CPU y no puede ser removido de ella de forma externa hasta finalizar.

Prioridad Expropiativa(Priority P)

Representa la implementación de máxima sensibilidad jerárquica. Es una política de corte expulsivo en la cual, ante el evento de entrada de un proceso de alta prioridad a la cola de listos, el planificador compara de inmediato su rango contra el del proceso en ejecución. Si el proceso entrante posee mayor urgencia jerárquica, la tarea en curso es expulsada sin contemplaciones, resguardando su estado.

Random (Aleatorio)

Política de planificación basada estrictamente en la probabilidad. En cada evento de toma de decisión (cuando la CPU queda libre o el proceso actual termina), el despachador ignora por completo cualquier heurística como el tiempo de llegada, la prioridad o la ráfaga restante, y simplemente selecciona un proceso de la cola de listos al azar mediante un generador estocástico.

Formulación Matemática de las Métricas de Rendimiento

Para auditar el desempeño de los planificadores de forma matemática y precisa, el simulador ejecuta ecuaciones deterministas en cada ciclo del reloj del sistema:

- **Tiempo de Retorno (Turnaround Time):** Mide el intervalo transcurrido desde el instante de arribo hasta el ciclo de reloj preciso donde completa su última instrucción de ráfaga.

Fórmula: $T_{\text{retorno}} = T_{\text{finalizacion}} - T_{\text{llegada}}$

- **Tiempo de Espera (Waiting Time):** Cuantifica el total de ciclos de reloj acumulados durante los cuales un proceso ha permanecido inactivo dentro de la cola de listos.

Fórmula: $T_{\text{espera}} = T_{\text{retorno}} - T_{\text{rafaga_cpu_total}} - T_{\text{bloqueo_io}}$

- **Tiempo de Respuesta (Response Time):** Registra la latencia existente entre la llegada inicial del proceso al sistema y el instante preciso en el cual la CPU le asigna su primer tick de ejecución.

Fórmula: $T_{\text{respuesta}} = T_{\text{primera_ejecucion}} - T_{\text{llegada}}$

- **Porcentaje de Utilización de CPU:** Proporción del tiempo total de la simulación en la cual el procesador se mantuvo ejecutando hilos activos.

Fórmula: $\%U_{\text{cpu}} = (\text{Suma } T_{\text{cpu_activo}} / T_{\text{total_simulacion}}) * 100$

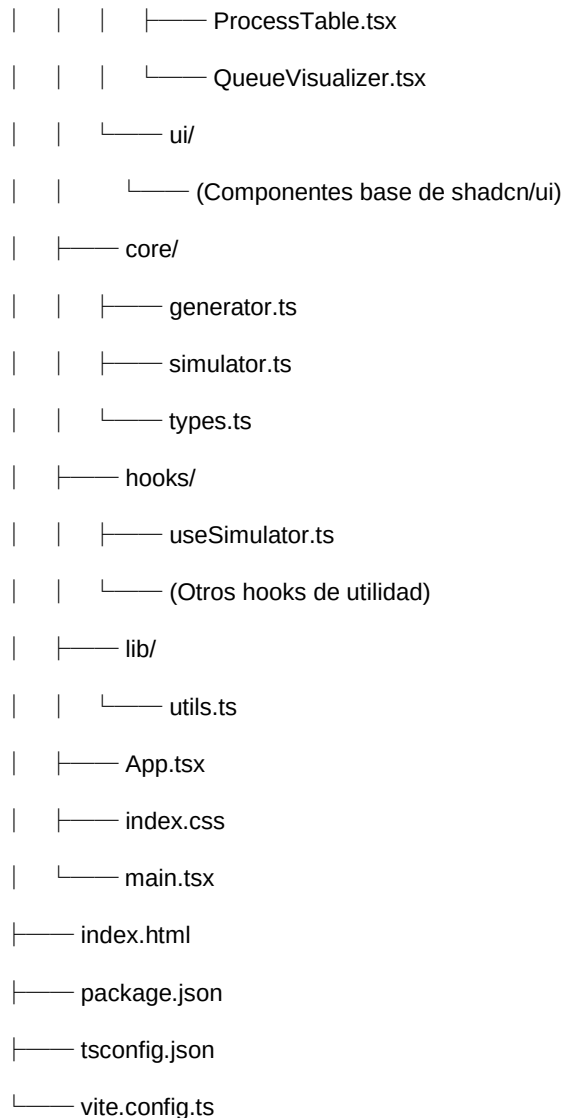
- **Rendimiento (Throughput):** Número neto de procesos completados exitosamente en el sistema por unidad de tiempo discreta.

Fórmula: $\text{Throughput} = \text{Procesos_Terminados} / T_{\text{total_simulacion}}$

Estructura del Proyecto

l-process-manager/

```
|— src/
|   |— components/
|   |   |— layout/
|   |   |   |— MainDashboard.tsx
|   |   |   |— Sidebar.tsx
|   |   |— simulation/
|   |   |— GanttVisualizer.tsx
```



src/core/ (Motor Lógico del Simulador): Contiene el cerebro matemático del simulador, completamente aislado de la interfaz visual.

- - `simulator.ts`: Contiene la clase y las funciones principales del simulador. Aquí se maneja el reloj del sistema (ticks), las transiciones de los procesos entre las colas (Listos, Ejecución, Bloqueados) y las implementaciones algorítmicas de las seis políticas de planificación (FCFS, SJF, SRTF, Round Robin, y Prioridades).
- - `generator.ts`: Archivo encargado de la lógica matemática para la generación de lotes de procesos aleatorios, creando casos de prueba con diferentes ráfagas de CPU, requerimientos de E/S y tiempos de llegada.
- - `types.ts`: Define las interfaces y tipados estrictos de TypeScript (como la estructura del Bloque de Control de Procesos o PCB), garantizando que los datos no sufran mutaciones inesperadas durante la ejecución.

src/hooks/ (Gestor de Estado y Sincronización):

- - useSimulator.ts: Este *Custom Hook* actúa como el puente de comunicación (middleware) entre el motor lógico y la interfaz gráfica. Se encarga de suscribir los componentes visuales a los ticks del reloj del simulador, actualizando el estado de la pantalla en tiempo real sin congelar la aplicación.

src/components/ (Interfaz Gráfica y Visualizadores): Contiene todos los elementos visuales con los que interactúa el usuario, subdivididos según su propósito:

- - simulation/: Contiene los visualizadores interactivos que le dan vida al simulador. Aquí se encuentra el GanttVisualizer.tsx (encargado de dibujar la línea de tiempo histórica del procesador), el QueueVisualizer.tsx (que renderiza el paso de los procesos por las colas) y el ProcessTable.tsx (tabla de métricas en tiempo real).
- - layout/: Componentes estructurales de la página, como el panel lateral de control (Sidebar.tsx) y el tablero principal (MainDashboard.tsx).
- - ui/: Una colección de componentes base reutilizables (botones, menús desplegables, tarjetas) contruidos para mantener un diseño limpio y consistente.

Archivos Raíz de Configuración (App.tsx, main.tsx, vite.config.ts): Orquestan la compilación del proyecto, sirviendo como los puntos de entrada de la aplicación web y definiendo las reglas de empaquetado para el entorno de producción.

Decisiones de Diseño y Desafíos Técnicos Superados

El mayor reto de implementación consistió en desacoplar el bucle de renderizado asíncrono inherente a React (basado en la reconciliación del Virtual DOM) del motor de simulación discreta por ticks. Si el estado global mutara directamente en cada iteración del bucle temporal, la interfaz experimentaría congelamientos o desfases temporales respecto al Diagrama de Gantt.

Para mitigar este inconveniente, se implementó un patrón de diseño basado en un Custom Hook centralizado (useSimulator) que opera de manera determinista. La simulación avanza sincronizada mediante un temporizador nativo que ejecuta una acción atómica por tick. En dicha acción se calculan los cambios en las estructuras de datos nativas de TypeScript, y posteriormente se actualiza el estado gráfico de una sola vez, garantizando una fluidez visual óptima.

Otro reto relevante fue el modelado realista de las operaciones de Entrada/Salida mixtas. El simulador lee vectores dinámicos que dictaminan si un proceso, tras completar ciertos ciclos de cómputo, debe abandonar el procesador de forma voluntaria para realizar tareas de E/S de longitud determinada. Esto permitió evaluar con total fidelidad el comportamiento de los algoritmos expulsivos ante escenarios con sobrecarga de bloqueos, replicando con exactitud los entornos de computación reales.